

✓ Lab 1. PyTorch and ANNs

This lab is a warm up to get you used to the PyTorch programming environment used in the course, and also to help you review and renew your knowledge of Python and relevant Python libraries. The lab must be done individually. Please recall that the University of Toronto plagiarism rules apply.

By the end of this lab, you should be able to:

1. Be able to perform basic PyTorch tensor operations.
2. Be able to load data into PyTorch
3. Be able to configure an Artificial Neural Network (ANN) using PyTorch
4. Be able to train ANNs using PyTorch
5. Be able to evaluate different ANN configurations

You will need to use numpy and PyTorch documentations for this assignment:

- <https://docs.scipy.org/doc/numpy/reference/>
- <https://pytorch.org/docs/stable/torch.html>

You can also reference Python API documentations freely.

What to submit

Submit a PDF file containing all your code, outputs, and write-up from parts 1-5. You can produce a PDF of your Google Colab file by going to `File -> Print` and then save as PDF. The Colab instructions has more information.

Do not submit any other files produced by your code.

Include a link to your colab file in your submission.

Please use Google Colab to complete this assignment. If you want to use Jupyter Notebook, please complete the assignment and upload your Jupyter Notebook file to Google Colab for submission.

Adjust the scaling to ensure that the text is not cutoff at the margins.

Colab Link

Submit make sure to include a link to your colab file here

Colab Link:

https://colab.research.google.com/github/sissi820li/APS360/blob/main/Lab1_PyTorch_and_ANNs.ipynb

✓ Part 1. Python Basics [3 pt]

The purpose of this section is to get you used to the basics of Python, including working with functions, numbers, lists, and strings.

Note that we **will** be checking your code for clarity and efficiency.

If you have trouble with this part of the assignment, please review <http://cs231n.github.io/python-numpy-tutorial/>

✓ Part (a) -- 1pt

Write a function `sum_of_cubes` that computes the sum of cubes up to `n`. If the input to `sum_of_cubes` is invalid (e.g. negative or non-integer `n`), the function should print out "Invalid input" and return `-1`.

```
def sum_of_cubes(n):
    """Return the sum (1^3 + 2^3 + 3^3 + ... + n^3)

    Precondition: n > 0, type(n) == int

    >>> sum_of_cubes(3)
    36
    >>> sum_of_cubes(1)
    1
    """

    if type(n) == int and n > 0:
        sum = 0
        while n > 0:
            sum += n**3
            n -= 1
        return sum

    print("Invalid input")
    return -1
```

✓ Part (b) -- 1pt

Write a function `word_lengths` that takes a sentence (string), computes the length of each word in that sentence, and returns the length of each word in a list. You can assume that words are always separated by a space character " ".

Hint: recall the `str.split` function in Python. If you aren't sure how this function works, try typing `help(str.split)` into a Python shell, or check out

<https://docs.python.org/3.6/library/stdtypes.html#str.split>

```
help(str.split)
```

Help on method_descriptor:

```
split(self, /, sep=None, maxsplit=-1) unbound builtins.str method
```

Return a list of the substrings in the string, using sep as the separator st

sep

The separator used to split the string.

When set to None (the default value), will split on any whitespace character (including \n \r \t \f and spaces) and will discard empty strings from the result.

maxsplit

Maximum number of splits.

-1 (the default value) means no limit.

Splitting starts at the front of the string and works to the end.

Note, `str.split()` is mainly useful for data that has been intentionally delimited. With natural text that includes punctuation, consider using the regular expression module.

```
def word_lengths(sentence):  
    """Return a list containing the length of each word in  
    sentence.  
  
    >>> word_lengths("welcome to APS360!")  
    [7, 2, 7]  
    >>> word_lengths("machine learning is so cool")  
    [7, 8, 2, 2, 4]  
    ""  
  
    lengths = []  
    if type(sentence) == str:  
        words = sentence.split()  
        for word in words:  
            length = len(word)  
            lengths.append(length)  
        return lengths
```

```
print ("Invalid input")
return -1
```

✓ Part (c) -- 1pt

Write a function `all_same_length` that takes a sentence (string), and checks whether every word in the string is the same length. You should call the function `word_lengths` in the body of this new function.

```
def all_same_length(sentence):
    """Return True if every word in sentence has the same
    length, and False otherwise.

    >>> all_same_length("all same length")
    False
    >>> word_lengths("hello world")
    True
    """

    if type(sentence) == str:
        if sentence == "" or sentence == " ":
            return True

        words = sentence.split()
        length = len(words[0])
        for word in words:
            if length != len(word):
                return False
        return True

    print ("Invalid input")
    return -1
```

✓ Part 2. NumPy Exercises [5 pt]

In this part of the assignment, you'll be manipulating arrays using NumPy. Normally, we use the shorter name `np` to represent the package `numpy`.

```
import numpy as np
```

✓ Part (a) -- 1pt

The below variables `matrix` and `vector` are numpy arrays. Explain what you think `<NumpyArray>.size` and `<NumpyArray>.shape` represent.

```
matrix = np.array([[1., 2., 3., 0.5],
                   [4., 5., 0., 0.],
                   [-1., -2., 1., 1.]])
vector = np.array([2., 0., 1., -2.])
```

```
#Returns the total number of elements in a matrix array
matrix.size
```

12

```
#Returns a tuple with the dimensions of the matrix array -> (number of rows, number of columns)
matrix.shape
```

(3, 4)

```
#Returns the total number of elements in a vector array
vector.size
```

4

```
#Returns a tuple showing the dimension/length of the vector array -> (length of vector)
vector.shape
```

(4,)

✓ Part (b) -- 1pt

Perform matrix multiplication `output = matrix x vector` by using for loops to iterate through the columns and rows. Do not use any builtin NumPy functions. Cast your output into a NumPy array, if it isn't one already.

Hint: be mindful of the dimension of output

```
output = None
```

```
def matrix_multiplication(matrix, vector):
    output = []
    for row in matrix:
        num = 0
        for i in range(vector.shape[0]):
            num += row[i]*vector[i]
        output.append(num)
```

```
    return np.array(output)

output = matrix_multiplication(matrix, vector)
```

✓ Part (c) -- 1pt

Perform matrix multiplication `output2 = matrix x vector` by using the function `numpy.dot`.

We will never actually write code as in part(c), not only because `numpy.dot` is more concise and easier to read/write, but also performance-wise `numpy.dot` is much faster (it is written in C and highly optimized). In general, we will avoid for loops in our code.

Start coding or [generate](#) with AI.

```
output2 = None
```

```
output2 = np.dot(matrix, vector)
```

✓ Part (d) -- 1pt

As a way to test for consistency, show that the two outputs match.

```
if np.array_equal(output, output2):
    print(True)
else:
    print(False)
```

```
True
```

✓ Part (e) -- 1pt

Show that using `np.dot` is faster than using your code from part (c).

You may find the below code snippet helpful:

```
import time

def measure_runtime(func, input1, input2):
    start_time = time.perf_counter()
    func(input1, input2)
```

```
end_time = time.perf_counter()
return (end_time - start_time)
```

```
measure_runtime(matrix_multiplication, matrix, vector)
```

```
2.480000011928496e-05
```

```
measure_runtime(np.dot, matrix, vector)
```

```
1.560000009703799e-05
```

✓ Part 3. Images [6 pt]

A picture or image can be represented as a NumPy array of “pixels”, with dimensions $H \times W \times C$, where H is the height of the image, W is the width of the image, and C is the number of colour channels. Typically we will use an image with channels that give the the Red, Green, and Blue “level” of each pixel, which is referred to with the short form RGB.

You will write Python code to load an image, and perform several array manipulations to the image and visualize their effects.

```
import matplotlib.pyplot as plt
```

✓ Part (a) -- 1 pt

This is a photograph of a dog whose name is Mochi.



Load the image from its url (https://drive.google.com/uc?export=view&id=1oaLVR2hr1_qzpKQ47i9rVUIklwbDcews) into the variable `img` using the `plt.imread` function.

Hint: You can enter the URL directly into the `plt.imread` function as a Python string.

```
import urllib.request
from PIL import Image

url = "https://drive.google.com/uc?export=view&id=1oaLVR2hr1_qzpKQ47i9rVUIklwbDcews"
img = np.array(Image.open(urllib.request.urlopen(url)))
```

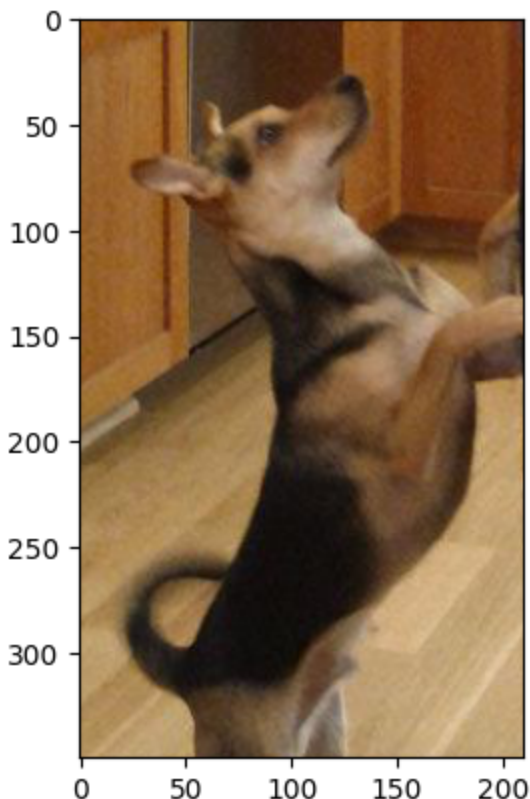
✓ Part (b) -- 1pt

Use the function `plt.imshow` to visualize `img`.

This function will also show the coordinate system used to identify pixels. The origin is at the top left corner, and the first dimension indicates the Y (row) direction, and the second dimension indicates the X (column) dimension.

```
plt.imshow(img)
```

<matplotlib.image.AxesImage at 0x1b2350c1160>

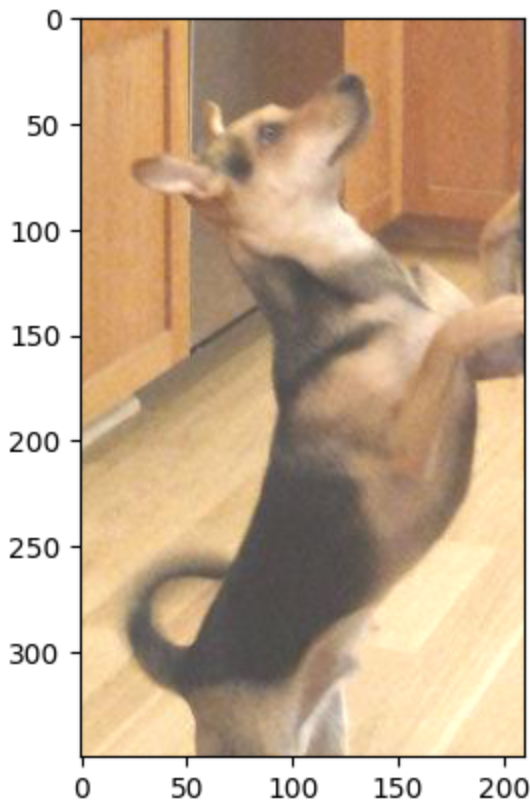


✓ Part (c) -- 2pt

Modify the image by adding a constant value of 0.25 to each pixel in the `img` and store the result in the variable `img_add`. Note that, since the range for the pixels needs to be between `[0, 1]`, you will also need to clip `img_add` to be in the range `[0, 1]` using `numpy.clip`. Clipping sets any value that is outside of the desired range to the closest endpoint. Display the image using `plt.imshow`.

```
img_normalized = img / 255.0
img_add = img_normalized + 0.25
img_add = np.clip(img_add, 0, 1)
plt.imshow(img_add)
```

<matplotlib.image.AxesImage at 0x1b23515a850>



✓ Part (d) -- 2pt

Crop the **original** image (`img` variable) to a 130 x 150 image including Mochi's face. Discard the alpha colour channel (i.e. resulting `img_cropped` should **only have RGB channels**)

Display the image.

```
img_cropped = img[0:130, 0:150, 0:3]  
plt.imshow(img_cropped)
```

<matplotlib.image.AxesImage at 0x1b2351dd310>



✓ Part 4. Basics of PyTorch [6 pt]

PyTorch is a Python-based neural networks package. Along with tensorflow, PyTorch is currently one of the most popular machine learning libraries.

PyTorch, at its core, is similar to Numpy in a sense that they both try to make it easier to write codes for scientific computing achieve improved performance over vanilla Python by leveraging highly optimized C back-end. However, compare to Numpy, PyTorch offers much better GPU support and provides many high-level features for machine learning. Technically, Numpy can be used to perform almost every thing PyTorch does. However, Numpy would be a lot slower than PyTorch, especially with CUDA GPU, and it would take more effort to write machine learning related code compared to using PyTorch.

```
import torch
```

✓ Part (a) -- 1 pt

Use the function `torch.from_numpy` to convert the numpy array `img_cropped` into a PyTorch tensor. Save the result in a variable called `img_torch`.

```
img_torch = torch.from_numpy(img_cropped)
```

✓ Part (b) -- 1pt

Use the method `<Tensor>.shape` to find the shape (dimension and size) of `img_torch`.

```
img_torch.shape
```

```
torch.Size([130, 150, 3])
```

✓ Part (c) -- 1pt

How many floating-point numbers are stored in the tensor `img_torch`?

```
shape = img_torch.shape
total = shape[0]*shape[1]*shape[2] #channels x height x width
print(str(total) + " numbers are stored in the tensor")
```

```
58500 numbers are stored in the tensor
```

✓ Part (d) -- 1 pt

What does the code `img_torch.transpose(0,2)` do? What does the expression return? Is the original variable `img_torch` updated? Explain.

```
#The command swaps the values of the 1st dimension of the tensor with the 3rd c
```

✓ Part (e) -- 1 pt

What does the code `img_torch.unsqueeze(0)` do? What does the expression return? Is the original variable `img_torch` updated? Explain.

```
#The command inserts a new dimension of size 1 to the tensor and shifts all exi
```

✓ Part (f) -- 1 pt

Find the maximum value of `img_torch` along each colour channel? Your output should be a one-dimensional PyTorch tensor with exactly three values.

Hint: lookup the function `torch.max`.

```
max_r = torch.max(img_torch[0])
max_g = torch.max(img_torch[1])
max_b = torch.max(img_torch[2])

max_values = torch.tensor([max_r, max_g, max_b])
```

✓ Part 5. Training an ANN [10 pt]

The sample code provided below is a 2-layer ANN trained on the MNIST dataset to identify digits less than 3 or greater than and equal to 3. Modify the code by changing any of the following and observe how the accuracy and error are affected:

- number of training iterations
- number of hidden units
- numbers of layers
- types of activation functions
- learning rate

Please select at least three different options from the list above. For each option, please select two to three different parameters and provide a table.

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision import datasets, transforms
import matplotlib.pyplot as plt # for plotting
import torch.optim as optim

torch.manual_seed(1) # set the random seed

# define a 2-layer artificial neural network
class Pigeon(nn.Module):
    def __init__(self):
        super(Pigeon, self).__init__()
        self.layer1 = nn.Linear(28 * 28, 30) #CHANGE # OF HIDDEN UNITS: 30 -> 1
        self.layer2 = nn.Linear(30, 1) #CHANGE # OF HIDDEN UNITS
    def forward(self, img):
```

```

        flattened = img.view(-1, 28 * 28)
        activation1 = self.layer1(flattened)
        activation1 = F.relu(activation1)
        activation2 = self.layer2(activation1)
        return activation2

pigeon = Pigeon()

# load the data
mnist_data = datasets.MNIST('data', train=True, download=True)
mnist_data = list(mnist_data)
mnist_train = mnist_data[:1000]
mnist_val = mnist_data[1000:2000]
img_to_tensor = transforms.ToTensor()

# simplified training code to train `pigeon` on the "small digit recognition" t
criterion = nn.BCEWithLogitsLoss()
optimizer = optim.SGD(pigeon.parameters(), lr=0.005, momentum=0.9) #CHANGE LEAF

for epoch in range(1): #CHANGE NUMBER OF EPOCHS: 1 -> 5, 10
    for (image, label) in mnist_train:
        # actual ground truth: is the digit less than 3?
        actual = torch.tensor(label < 3).reshape([1,1]).type(torch.FloatTensor)
        # pigeon prediction
        out = pigeon(img_to_tensor(image)) # step 1-2
        # update the parameters based on the loss
        loss = criterion(out, actual) # step 3
        loss.backward() # step 4 (compute the updates for ea
        optimizer.step() # step 4 (make the updates for each
        optimizer.zero_grad() # a clean up step for PyTorch

# computing the error and accuracy on the training set
error = 0
for (image, label) in mnist_train:
    prob = torch.sigmoid(pigeon(img_to_tensor(image)))
    if (prob < 0.5 and label < 3) or (prob >= 0.5 and label >= 3):
        error += 1
print("Training Error Rate:", error/len(mnist_train))
print("Training Accuracy:", 1 - error/len(mnist_train))

# computing the error and accuracy on a test set
error = 0
for (image, label) in mnist_val:
    prob = torch.sigmoid(pigeon(img_to_tensor(image)))
    if (prob < 0.5 and label < 3) or (prob >= 0.5 and label >= 3):
        error += 1
print("Test Error Rate:", error/len(mnist_val))
print("Test Accuracy:", 1 - error/len(mnist_val))

```

Training Error Rate: 0.036
 Training Accuracy: 0.964
 Test Error Rate: 0.079
 Test Accuracy: 0.921

1. Change the number of Hidden units:

# of Hidden Units	Training Error	Training Accuracy	Testing Error	Testing Accuracy
10	0.047	0.953	0.104	0.896
30	0.036	0.964	0.079	0.921
100	0.033	0.967	0.074	0.926
500	0.030	0.970	0.077	0.923

2. Change the Learning rate:

Learning Rate	Training Error	Training Accuracy	Testing Error	Testing Accuracy
0.001	0.078	0.922	0.113	0.887
0.005	0.036	0.964	0.079	0.921
0.050	0.312	0.688	0.297	0.703
0.100	0.312	0.688	0.297	0.703

3. Change the number of Epochs:

# of Epochs	Training Error	Training Accuracy	Testing Error	Testing Accuracy
1	0.036	0.964	0.079	0.921
5	0.011	0.989	0.066	0.934
10	0.001	0.999	0.059	0.941
15	0.001	0.999	0.059	0.941

✓ Part (a) -- 3 pt

Comment on which of the above changes resulted in the best accuracy on training data?
 What accuracy were you able to achieve?

#The best result was obtained by increasing the number of iterations/epochs. Wh

✓ Part (b) -- 3 pt

Comment on which of the above changes resulted in the best accuracy on testing data?
 What accuracy were you able to achieve?

```
#Similar to the training data, the best result was obtained by increasing the r
```

✓ Part (c) -- 4 pt

Which model hyperparameters should you use, the ones from (a) or (b)?

```
#The model hyperparameters should always prioritize the results from the testir
```